Data Structures and Algorithms -3 (25CS2103E)

Network Flow Optimizer

Team Members

Sangu Nikhil - 2520030246

V N V Sai Chandu - 2520030522

CH. Pavan kumar - 2520030312

Under the Esteemed Guidance of

Dr. CH. Anuradha

Department of CSE

Contents

- Abstract
- Introduction
- Literature survey
- Problem Statement and Objectives
- Hardware and Software requirements
- Results
- Conclusion
- Future Scope

Abstract

NetworkFlowOptimizer is a Java application that solves the maximum-flow problem in a directed graph. Each edge has a capacity, and the objective is to send the greatest possible amount from a source to a sink without exceeding any edge capacity.

The project implements Dinic's Algorithm and Edmonds–Karp Algorithm and provides a Java Swing interface to run and compare them.

On the supplied six-vertex example network, both algorithms should return a maximum flow of 23 units.

Introduction

Network flow models how material, data or resources move through a network of connected points.

A graph contains vertices (nodes) and directed edges. Each edge has a capacity that limits how much flow it can carry.

The source is vertex 0 and the sink is vertex 5 in the sample program.

The application demonstrates graph representation, residual networks, maximum-flow algorithms and execution-time comparison.

Literature Survey

Maximum-flow algorithms solve the problem of finding the greatest feasible flow from a source to a sink in a capacitated directed graph.

Ford–Fulkerson introduced the augmenting-path approach.

Edmonds–Karp uses breadth-first search to select shortest augmenting paths and has $O(VE^2)$ time complexity.

Dinic's Algorithm builds a level graph with BFS and sends blocking flow with DFS; its standard general-graph bound is $O(V^2E)$.

This project compares the two approaches using the same sample network.

Problem Statement and Objectives

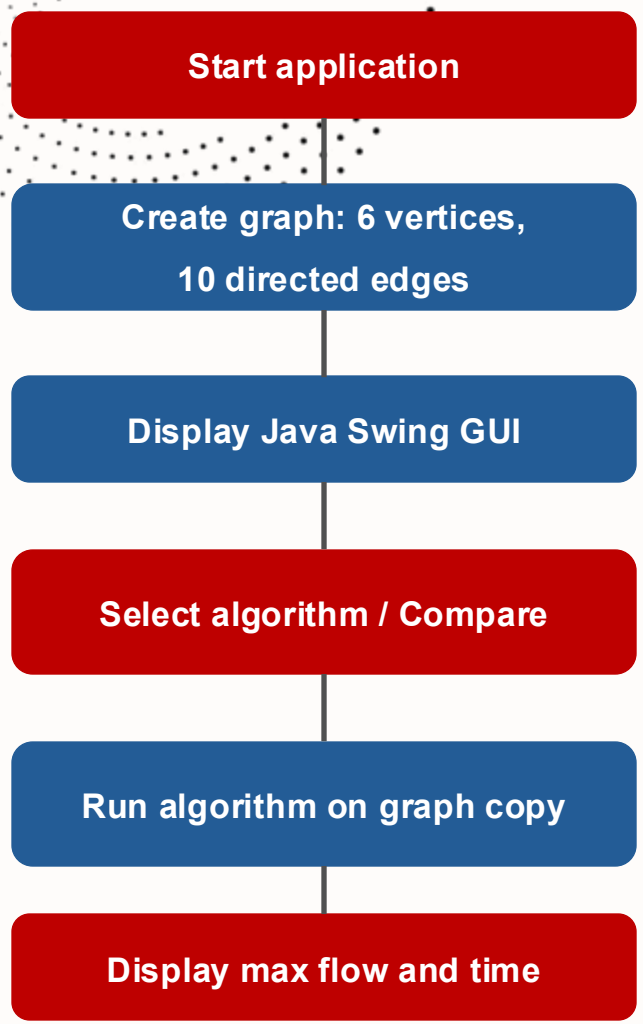
Problem statement:

Given a directed graph with edge capacities, calculate the maximum feasible flow from source vertex 0 to sink vertex 5.

Objectives:

- Represent vertices, edges, capacities, flow and reverse edges in Java.
- Implement Dinic's Algorithm and Edmonds–Karp Algorithm.
- Build a Java Swing GUI to execute algorithms and display results.
- Compare maximum-flow values and execution times.

Flow Chart



Algorithm execution

- Dinic: BFS builds a level graph; DFS sends blocking flow.
- Edmonds–Karp: BFS finds one augmenting path at a time.
- Residual capacity = capacity − current flow.
- Reverse edges allow previous flow to be adjusted.
- Stop when no augmenting route to the sink remains.

Results

Sample graph: 6 vertices (0–5) and 10 directed edges.

Source = 0 | Sink = 5

Dinic maximum flow: 23 units

Edmonds–Karp maximum flow: 23 units

Both algorithms should return the same maximum-flow value. Runtime depends on machine load, JVM warm-up and graph structure; a larger repeated benchmark is needed for a reliable speed comparison.

Hardware and Software requirements

Hardware requirements

- Computer or laptop
- Minimum 4 GB RAM recommended
- Keyboard, mouse and display

Software requirements

- Java Development Kit (JDK 17 or later recommended)
- Visual Studio Code or another Java IDE
- Java extension pack for VS Code (if using VS Code)
- Operating system: macOS, Windows or Linux

Conclusion

The project demonstrates how a maximum-flow problem can be solved using graph algorithms in Java.

Graph, Vertex and Edge classes model the network and residual capacities.

Dinic and Edmonds–Karp provide two approaches to computing maximum flow.

The Swing interface makes it possible to run the algorithms and compare their outputs.

The sample network produces a maximum flow of 23 units.

Future Scope

Allow users to create and edit graphs instead of using a fixed example.

Allow configurable source and sink vertices.

Display the actual flow on each edge and animate algorithm steps.

Implement a functional Reset button and improve validation for invalid inputs.

Benchmark larger generated networks over multiple runs and report average execution time.

Add automated tests for known maximum-flow cases.

(DEEMED TO BE UNIVERSITY)

Thank you